

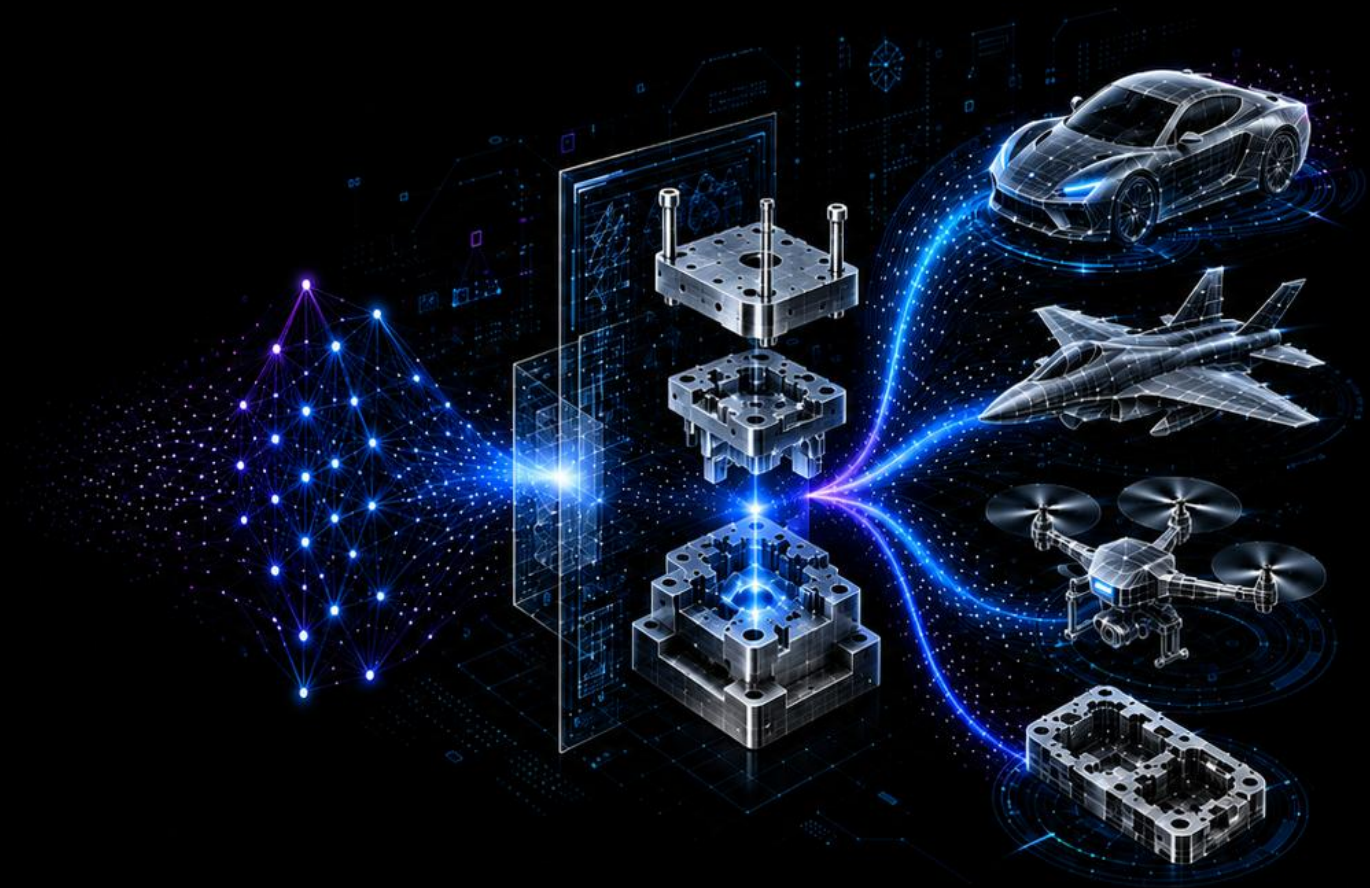
개인 과제 발표

Text2CAD – 4

최준석

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design
for Engineering
Applications Lab

■ 진행 상황

- ❖ Kimi와 Qwen 모델의 차이점 -> 완료
- ❖ 전체적인 코드 정리(+ Rule-base 여부 포함) -> 완료
- ❖ RAG를 깼다가 킬 수 있는 작업 -> 완료
- ❖ Skill 추가입력
- ❖ 항공기 형상 진행상황 -> 진행 중
- ❖ Build123 vs freeCAD 비교 -> 항공기 형상 완료 후 진행할 예정

■ 현재 진행 방향

kimi

- ❖ Kimi의 특징은 반복적이고 정형화된 작업 절차 중심입니다.
- ❖ 단기적으로 봤을 때는 여러 skill들을 활용해서 결과물을 확보하기엔 적절하다고 생각
- ❖ 단점은 해당 모델을 썼을 경우 Local LLM을 포기해야한다는 단점

Qwen

- ❖ Kimi 보다는 모델이 능력이 낮다
- ❖ 하지만 Local LLM을 쓸 수 있는 최적의 모델
- ❖ 해당 모델의 특징은 현재 진행하고 있는 LLM이 자연어를 받아 FreeCAD 코드를 작성한다에 가장 부합
 - ✓ 'Agent + Tools + Instruction' 방식
- ❖ 범용적인 시스템을 구축 가능

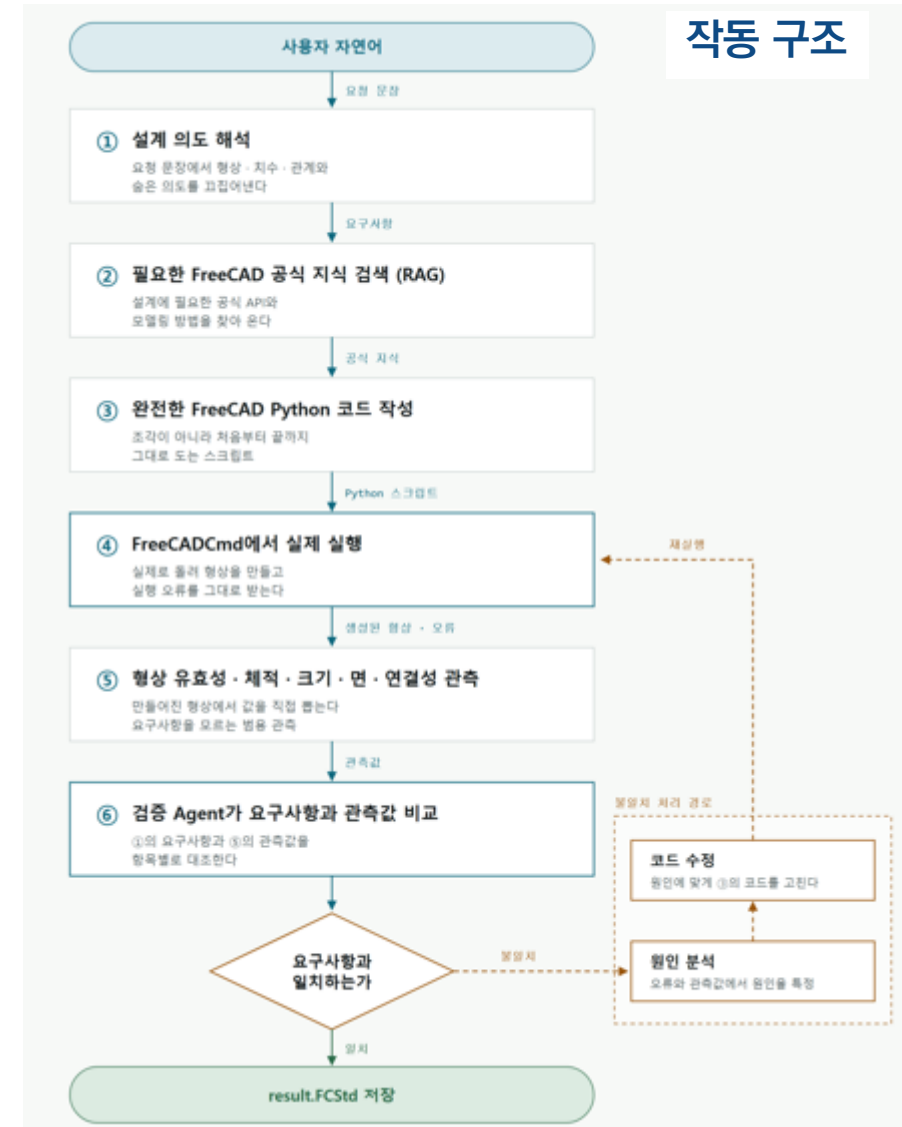
전체적인 흐름

연구 목표 & 방향

- ❖ 사용자가 입력한 자연어를 Local LLM이 직접 이해하고, 필요한 FreeCAD 지식을 RAG에서 찾아 Python 코드를 작성한 뒤 실제 CAD 형상을 생성·검증·수정하는 시스템
- ❖ 최종 목표는 특정 부품만 만드는 프로그램이 아니라, 처음 보는 자연어 요청과 다양한 형상에도 적용할 수 있는 범용 Text-to-CAD 시스템

핵심 원칙

- ✓ 자연어 해석, 설계 방법 선택, 코드 작성, 오류 수정은 LLM이 수행
- ✓ RAG는 완성품 정답이 아니라 공식 FreeCAD 사용 지식을 제공
- ✓ 특정 부품 전용 생성기 사용 금지
- ✓ 기존 STEP 파일은 평가에만 사용하고 생성 과정에는 제공하지 않음
- ✓ 실행 성공과 설계 정확도를 구분하여 평가



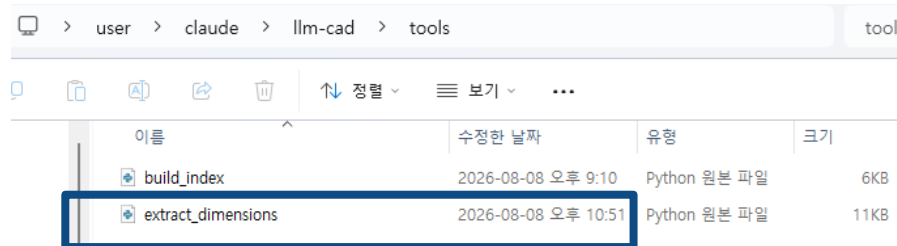
■ 각 구성요소의 역할

설계 해석 LLM

❖ 사용자의 문장을 단순 키워드로 분류하지 않고 다음 내용을 자연어 문맥에서 해석

- ✓ 전체 형상
- ✓ 주요 치수
- ✓ 형상 간 위치와 연결 관계
- ✓ 구멍, 슬롯, 단차, 모따기, 필렛 등의 특징
- ✓ 가능한 모델링 순서
- ✓ 불명확하거나 추가 지식이 필요한 부분

CAD 파일 → 자연어



이름	수정된 날짜	유형	크기
build_index	2026-08-08 오후 9:10	Python 원본 파일	6KB
extract_dimensions	2026-08-08 오후 10:51	Python 원본 파일	11KB

STEP-FCStd 파일



extract_dimensions.py



형상의 객관적인 치수와 기하 정보를 Markdown으로 추출



reference/dimensions/부품명.md



별도의 LLM이 치수표를 읽고 자연어 요청문 작성



reference/dev 또는 holdout/부품명.txt

■ 각 구성요소의 역할

RAG

- ❖ LLM이 설계를 구현하기 위해 필요한 FreeCAD 지식을 공식 문서에서 검색
- ❖ 현재 RAG는 다음과 같은 지식을 제공하는 역할입니다.
 - ✓ 기본 형상 생성
 - ✓ 스케치, 와이어, 면, 돌출 및 회전
 - ✓ Boolean 결합과 절삭
 - ✓ 구멍, 슬롯, 필렛, 모따기
 - ✓ 모서리와 면의 속성 확인
 - ✓ FreeCAD 객체 생성과 문서 저장
 - ✓ 오류가 발생한 API의 올바른 사용법
- ❖ RAG에는 특정 부품의 완성 코드나 “브라켓은 이 코드로 만든다”와 같은 레시피를 넣지 않는 것이 원칙으로 설정

코드 생성 LLM

- ❖ 해석 결과와 RAG 지식을 조합하여 완전한 FreeCAD Python 코드를 작성한다
- ❖ 실제 형상의 구성 방법과 연산 순서는 LLM이 결정합니다.

■ 각 구성요소의 역할

검증·수정 Agent

- ❖ 다음 세 가지 상태로 결과를 판단.
 - ✓ satisfied: 요청의 조건이 코드와 실행 증거로 확인됨
 - ✓ violated: 요청과 결과 사이에 확인 가능한 모순이 있음
 - ✓ uncertain: 형상은 생성됐지만 현재 증거만으로 정확성을 확증할 수 없음
- ❖ violated인 경우 LLM이 오류 원인과 부족한 지식을 검토하고 코드를 다시 작성합니다

■ Rule-base 여부

사용하지 않는 방식

- ❖ 현재 기본 파이프라인은 형상별 Rule-base 방식이 아님
 - ✓ "bracket"이라는 단어가 있으면 브라켓 생성기 실행
 - ✓ "hole"이 있으면 구멍 함수 호출
 - ✓ 형상 종류별 if/else
 - ✓ 키워드·정규식·동의어 사전 기반 자연어 해석
 - ✓ 브라켓, 축, 와셔별 완성 코드 템플릿
 - ✓ 평가 문제의 치수나 좌표를 숨은 기본값으로 사용
 - ✓ 기준 STEP을 생성·수정 프롬프트에 제공

사용하는 결정론적 코드

- ❖ 다음 기능은 일반 프로그램 제어 및 관측 기능이며 설계를 대신하지 않는다.
 - ✓ FreeCAD Python 실행
 - ✓ STEP/FCStd 로드
 - ✓ JSON 형식 검사
 - ✓ 체적·경계상자·솔리드·면·원통 측정
 - ✓ 수치 오차 판정
 - ✓ 파일 저장과 실행 오류 수집

⇒ 즉, 형상의 의미는 LLM이 판단하고 프로그램은 결과를 관측하는 구조

■ Skill은 Rule-base인가?

추가한 내용

- ❖ 최근 다음 세 가지 범용 CAD 사고 절차를 추가
 - ✓ 01_cad_brief.md: 요청을 치수·공간관계·불확실성·구성 단계로 정리
 - ✓ 02_evidence_validation.md: 코드, 실행 결과, 실제 측정값을 구분하여 검증
 - ✓ 03_repair_loop.md: 확인된 오류를 보존된 코드에서 최소 수정

⇒ Skill은 형상 이름으로 선택되지 않습니다.

Skill ON/OFF 실험 결과

- ❖ 환경: NVIDIA L4 24GB, Qwen3.5 27B Q4, RAG ON, Vision/Thinking OFF
- ❖ Skill OFF: 유효 평가 4건 중 성공 0건, 5번째 사례에서 실험 중단
- ❖ Skill ON: 첫 사례 실패 후 두 번째 channel_u에서 기준과 거의 동일한 형상 생성
 - ✓ 체적 오차 약 0.002%
 - ✓ 외형 크기·솔리드 수·원통 수 일치
- ❖ 하지만 검증 Agent의 근거 없는 면 개수 추정과 프로그램 예외로 전체 실험 중단

⇒ **결론:** Skill 적용 시 정확한 형상 생성 가능성은 확인했지만,
두 조건 모두 완주하지 못해 성능 향상이 통계적으로 입증된 것은 아니다.

■ 복잡한 형상을 생성하는 원리

원리

- ❖ 아직 브라켓과 같은 복잡한 형상은 미시도
- ❖ 현재는 와셔나 파이프 구조만 실행중
- ❖ 만약 실험한다면 브라켓은 LLM이 자연어를 통해 다음과 같이 해석
 - ✓ 바닥판과 수직판을 각각 생성
 - ✓ 두 판을 결합
 - ✓ 지정된 위치에 원형 구멍이나 장공을 절삭
 - ✓ 연결부와 외곽에 필렛 또는 모따기 적용
- ❖ 중요한 점은 이러한 순서를 브라켓 전용 규칙이 선택하는 것이 아니라, LLM이 요청과 RAG 지식을 바탕으로 매번 설계한다는 것이다.
- ❖ 따라서 이론적으로 다양한 형상을 표현할 수 있지만, 브라켓을 포함한 복잡 형상을 항상 정확히 생성하는 범용성은 아직 완전히 입증되지 않음.



■ RAG의 역할

주요 지식

❖ RAG에는 완성 부품 레시피가 아니라 공식 FreeCAD 문서

- ✓ FreeCAD scripting 기본
- ✓ Part 모듈
- ✓ Primitive
- ✓ Extrude, Revolve, Loft, Sweep
- ✓ Boolean Fuse/Cut/Common
- ✓ Fillet, Chamfer, Thickness
- ✓ Placement와 Vector
- ✓ B-rep 및 위상 구조
- ✓ 형상 유효성 검사

동작 방식

1. LLM이 요청과 설계 해석을 보고 검색 질의를 작성합니다.
2. nomic-embed-text로 공식 문서 인덱스를 검색합니다.
3. 검색된 API 근거를 코드 생성 프롬프트에 제공합니다.
4. 실행 오류가 발생하면 오류와 관련된 지식을 다시 검색합니다.

❖ Python 코드가 요청 키워드를 보고 문서를 선택하지 않으므로 의미는 계속 LLM이 담당

■ 주요 파일 구성

bench	2026-08-08 오후 3:22	파일 폴더	
guards	2026-08-08 오후 3:23	파일 폴더	
knowledge	2026-08-08 오후 9:11	파일 폴더	
llmcad	2026-08-08 오후 9:12	파일 폴더	
out	2026-08-08 오후 10:34	파일 폴더	
reference	2026-08-08 오후 4:29	파일 폴더	
results	2026-08-09 오후 2:02	파일 폴더	
runs	2026-08-08 오후 9:36	파일 폴더	
tools	2026-08-08 오후 9:10	파일 폴더	
bench	2026-08-08 오후 6:23	Python 원본 파일	6KB
CLAUDE	2026-08-08 오후 10:32	MD 파일	14KB
compare_reference	2026-08-08 오후 10:35	Python 원본 파일	12KB
HANDOFF	2026-08-09 오후 2:11	MD 파일	38KB
make_cad	2026-08-08 오후 3:12	Python 원본 파일	4KB

__pycache__	2026-08-17 오후 3:55	파일 폴더	
__init__	2026-08-08 오후 3:07	Python 원본 파일	1KB
cad	2026-08-08 오후 6:00	Python 원본 파일	11KB
config	2026-08-17 오후 3:55	Python 원본 파일	8KB
llm	2026-08-17 오후 3:55	Python 원본 파일	19KB
loop	2026-08-17 오후 2:35	Python 원본 파일	25KB
prompts	2026-08-17 오후 2:53	Python 원본 파일	18KB
retrieve	2026-08-14 오후 8:31	Python 원본 파일	5KB
score	2026-08-09 오후 3:49	Python 원본 파일	4KB
vision	2026-08-17 오후 2:53	Python 원본 파일	9KB

```
llm-cad/
|
├─ make_cad.py
|   └─ 사용자 요청을 받아 전체 생성 과정을 시작하는 실행 파일
|
├─ llmcad/
|   ├── config.py
|   |   └─ 모델, FreeCAD 경로, 반복 횟수, RAG·Vision 설정
|   ├── llm.py
|   |   └─ ollama 및 외부 LLM 호출, 응답 처리
|   ├── prompts.py
|   |   └─ 설계 해석, RAG 질문, 코드 생성, 검증·수정 프롬프트
|   ├── retrieve.py
|   |   └─ 공식 문서 임베딩 검색 및 검색 결과 전달
|   ├── loop.py
|   |   └─ 해석→검색→생성→실행→검증→수정의 전체 반복 제어
|   ├── cad.py
|   |   └─ FreeCAD 코드 실행, FCStd 저장, 일반 형상 정보 측정
|   ├── score.py
|   |   └─ 생성 결과와 기존 형상의 수치 비교
|   └─ vision.py
|       └─ CAD를 여러 방향에서 렌더링하고 시각 관측하는 실험 기능
```

■ 주요 파일 구성

bench	2026-08-08 오후 3:22	파일 폴더	
guards	2026-08-08 오후 3:23	파일 폴더	
knowledge	2026-08-08 오후 9:11	파일 폴더	
llmcad	2026-08-08 오후 9:12	파일 폴더	
out	2026-08-08 오후 10:34	파일 폴더	
reference	2026-08-08 오후 4:29	파일 폴더	
results	2026-08-09 오후 2:02	파일 폴더	
runs	2026-08-08 오후 9:36	파일 폴더	
tools	2026-08-08 오후 9:10	파일 폴더	
bench	2026-08-08 오후 6:23	Python 원본 파일	6KB
CLAUDE	2026-08-08 오후 10:32	MD 파일	14KB
compare_reference	2026-08-08 오후 10:35	Python 원본 파일	12KB
HANDOFF	2026-08-09 오후 2:11	MD 파일	38KB
make_cad	2026-08-08 오후 3:12	Python 원본 파일	4KB

index	2026-08-09 오후 3:42	파일 폴더	
official	2026-08-09 오후 3:44	파일 폴더	
freecad_api	2026-08-07 오후 10:33	MD 파일	
__pycache__	2026-08-17 오후 4:19	파일 폴더	
build_index	2026-08-14 오후 8:31	Python 원본 파일	10KB
compare_geometry	2026-08-09 오후 5:27	Python 원본 파일	3KB
extract_dimensions	2026-08-09 오후 5:28	Python 원본 파일	12KB

```
└ knowledge/
  └ 공식 FreeCAD 문서 자료
    └ index/
      └ vectors.npy
      └ chunks.jsonl

└ tools/
  └ build_index.py
    └ 공식 문서를 RAG 검색용 색인으로 변환
```

■ 주요 파일 구성

bench	2026-08-08 오후 3:22	파일 폴더	
guards	2026-08-08 오후 3:23	파일 폴더	
knowledge	2026-08-08 오후 9:11	파일 폴더	
llmcad	2026-08-08 오후 9:12	파일 폴더	
out	2026-08-08 오후 10:34	파일 폴더	
reference	2026-08-08 오후 4:29	파일 폴더	
results	2026-08-09 오후 2:02	파일 폴더	
runs	2026-08-08 오후 9:36	파일 폴더	
tools	2026-08-08 오후 9:10	파일 폴더	
bench	2026-08-08 오후 6:23	Python 원본 파일	6KB
CLAUDE	2026-08-08 오후 10:32	MD 파일	14KB
compare_reference	2026-08-08 오후 10:35	Python 원본 파일	12KB
HANDOFF	2026-08-09 오후 2:11	MD 파일	38KB
make_cad	2026-08-08 오후 3:12	Python 원본 파일	4KB

answers	2026-08-09 오후 3:42	파일 폴더	
requests	2026-08-09 오후 3:42	파일 폴더	
dev	2026-08-09 오후 3:42	파일 폴더	
dimensions	2026-08-09 오후 3:42	파일 폴더	
holdout	2026-08-09 오후 3:42	파일 폴더	
verification	2026-08-09 오후 5:27	파일 폴더	
README	2026-08-08 오후 4:18	MD 파일	

```
└─ reference/
  │   └─ dev/
  │       └─ 개발 중 반복 평가하는 형상과 자연어 요청
  │   └─ holdout/
  │       └─ 개발에 사용하지 않고 최종 일반화를 확인하는 시험 세트
  │   └─ dimensions/
  │       └─ 기준 형상의 측정 정보
  │
  └─ compare_reference.py
      └─ 생성 형상과 기준 STEP 파일을 비교하는 평가 도구
  │
  └─ bench.py
      └─ 여러 자연어 요청을 반복 실행하는 벤치마크 도구
```

■ 주요 파일 구성

bench	2026-08-08 오후 3:22	파일 폴더	
guards	2026-08-08 오후 3:23	파일 폴더	
knowledge	2026-08-08 오후 9:11	파일 폴더	
llmcd	2026-08-08 오후 9:12	파일 폴더	
out	2026-08-08 오후 10:34	파일 폴더	
reference	2026-08-08 오후 4:29	파일 폴더	
results	2026-08-09 오후 2:02	파일 폴더	
runs	2026-08-08 오후 9:36	파일 폴더	
tools	2026-08-08 오후 9:10	파일 폴더	
bench	2026-08-08 오후 6:23	Python 원본 파일	6KB
CLAUDE	2026-08-08 오후 10:32	MD 파일	14KB
compare_reference	2026-08-08 오후 10:35	Python 원본 파일	12KB
HANDOFF	2026-08-09 오후 2:11	MD 파일	38KB
make_cad	2026-08-08 오후 3:12	Python 원본 파일	4KB

```
└─ runs/.work/
  └─ 생성 코드, LLM 응답, 오류, 측정값과 FCStd 결과 보관
└─ CLAUDE.md
  └─ 연구 목표와 rule-base 금지 원칙
└─ HANDOFF.md
  └─ 구조, 실행법, 실험 기록과 인수인계 정보
```

■ 주요 파일 구성 (새롭게 추가된 자료)

__pycache__	2026-08-17 오후 3:55	파일 폴더	
baseline	2026-08-18 오전 12:33	파일 폴더	
bench	2026-08-09 오후 3:42	파일 폴더	
guards	2026-08-14 오후 8:32	파일 폴더	
knowledge	2026-08-09 오후 3:44	파일 폴더	
llmcd	2026-08-17 오후 2:33	파일 폴더	
out	2026-08-09 오후 3:48	파일 폴더	
reference	2026-08-09 오후 5:20	파일 폴더	
results	2026-08-09 오후 3:42	파일 폴더	
runs	2026-08-09 오후 4:41	파일 폴더	
skills	2026-08-17 오후 9:03	파일 폴더	
tools	2026-08-18 오전 12:44	파일 폴더	
work	2026-08-09 오후 9:47	파일 폴더	
bench	2026-08-17 오후 2:34	Python 원본 파일	7KB
CLAUDE	2026-08-17 오후 9:52	MD 파일	19KB
colab-rag-toggle-rules-v7	2026-08-17 오후 1:35	압축(ZIP) 폴더	32KB
colab-thinking-toggle-v10	2026-08-17 오후 3:55	압축(ZIP) 폴더	13KB
compare reference	2026-08-17 오후 2:34	Python 원본 파일	13KB
crm-wing-inspection-colab	2026-08-18 오전 12:45	압축(ZIP) 폴더	10,506KB
HANDOFF	2026-08-17 오후 2:35	MD 파일	40KB
make_cad	2026-08-17 오후 3:55	Python 원본 파일	6KB
requirements	2026-08-17 오후 2:35	텍스트 문서	1KB

❖ skills

- ✓ 범용 설계·검증·수정 사고 절차

❖ CRM 자료

- ✓ NASA 공식 CRM 전체 항공기 STEP
- ✓ NASA 공식 좌측 날개 STEP
- ✓ 출처·스케일·SHA-256 기록

■ 현재 실험 환경 + RAG on/off 기능

이전

- 생성 모델: `qwen3.5:27b`
- 실행 환경: Google Colab A100 40GB
- LLM 실행: Ollama 기반 Local LLM
- 임베딩 모델: `nomic-embed-text`
- RAG: 기본 ON, 사용자가 직접 ON/OFF 가능
- Vision: 기본 OFF, 사용자가 직접 ON/OFF 가능
- FreeCAD: GUI가 아닌 `FreeCADCmd` 로 자동 실행

현재

24GB VRAM 환경 기준:

- GPU: NVIDIA L4, 약 23GB
- 모델: `qwen3.5:27b-q4_K_M`
- 양자화: Q4
- Context: 16K
- RAG: ON
- Vision: OFF
- Thinking: OFF
- 출력 상한: 2,048 tokens
- 수정 회차: 최대 4회

- ❖ 현재 vision 같은 경우에는 LLM에게 형상을 보여주고 정확도를 높이는 방향으로 가려 했으나 큰 차이가 없어 보류 중
- ❖ VRAM 환경 저하로 모델의 양자화 방식 Q8에서 Q4로 변경
 - ✓ 그로 인해 기존 모델보다 코드 작성과 정밀 추론 기능 낮아짐

■ 현재 성과 & 핵심 문제

현재까지 확인된 성과

- 자연어 입력부터 FreeCAD `.FCStd` 생성까지 이어지는 전체 자동화 파이프라인 구축
- Local LLM이 설계 해석, 공식 FreeCAD RAG 검색, Python 코드 생성과 수정 판단 수행
- 형상별 생성기·키워드 분기 없이 범용 Agent 구조 유지
- FreeCAD에서 코드를 직접 실행하고 오류·체적·크기·솔리드·면 정보를 자동 관측
- 실행 실패 시 traceback과 형상 증거를 이용하는 반복 수정 루프 구현
- RAG ON/OFF, Vision ON/OFF, Skill ON/OFF를 분리 비교할 수 있는 실험 환경 구축
- 24GB VRAM에서 `Qwen3.5 27B Q4` 와 16K context 실행 가능성 확인
- Skill ON 실험에서 `channel_u` 형상을 외형·솔리드 수 일치, 체적 오차 약 `**0.002%**`로 생성
- 기존 STEP 유출 방지, dev/holdout 분리, 평가자료 자체 검증 체계 구축
- NASA 공식 CRM 전체 항공기 및 좌측 날개 STEP 확보와 출처·해시 기록 완료
- CRM 날개부터 전체 항공기 및 메싱 적합 형상으로 확장하는 baseline/ablation 연구 절차 수립

❖ 단순 CAD 생성 데모를 넘어, Local LLM이 공식 지식을 검색하고 실제 형상을 실행·관측·수정하는 범용 연구 플랫폼을 구축했으며, 현재 항공기 형상으로 연구 범위를 확장하는 단계

현재까지 확인된 한계

- LLM이 자연어를 제대로 이해해도 좌표 계산을 잘못할 수 있음
- 유효한 솔리드라고 해서 요구 형상이 정확한 것은 아님
- bbox와 체적만으로 곡면 위치와 연결관계를 충분히 확인하기 어려움
- 검증 Agent가 실제 증거보다 자신이 예상한 면 개수를 신뢰하는 경우가 있음
- 긴 응답이 출력 상한에서 잘리면 코드 블록 규약을 위반함
- FreeCAD 모듈 import 방식 같은 단순 API 오류가 발생함
- Vision ON은 현재 정확도를 높이지 못해 보류
- Skill 효과는 아직 정상적인 완주 A/B로 검증되지 않음

❖ 다음 범용 개선 후보는 B-rep의 면 종류, 곡면 파라미터, 위치, 면 인접관계를 Agent에게 증거로 제공하는 것

■ 연구 자료 및 진행 단계

공식 자료

- NASA CRM 전체 항공기 STEP: 풀 스케일
- NASA CRM 좌측 날개 STEP: 모델 스케일
- 두 파일은 스케일이 다르므로 직접 비교 전에 정규화 필요
- STEP에는 최종 B-rep은 있지만 CATIA 설계 이력이나 Python 코드는 없음

진행 단계

❖ 1단계: 공식 형상 분석

- STEP을 최신 FreeCAD로 가져오기
- 객체·솔리드·셀·면·모서리 확인
- B-spline 곡면, 단면 변화, 면 연결관계 추출
- 전체 항공기와 별도 날개의 스케일 관계 확정

❖ 현재는 이 단계입니다. Colab의 FreeCAD 0.19에서 STEP import가 segmentation fault를 일으켜 FreeCAD 1.1.3으로 교체했고, 상세 검사가 너무 오래 걸려 잠시 중단한 상태

■ 연구 진행 단계

진행 단계

❖ 2단계: 날개 baseline 구축

- 공식 CRM 단면과 외형 분석
- 루트·중간·팁 단면 구성
- 스펠 방향 배치
- 테이퍼·후퇴각·비틀림 적용
- FreeCAD Loft 기반 Python 코드 작성
- 공식 날개 STEP과 형상 비교

진행 단계

❖ 3단계: 전체 CRM 구성

- 동체
- 주익
- 수평 꼬리날개
- 수직 꼬리날개
- 페어링 및 접합부
- 최종 Compound 또는 단일 외부 형상 구성

- ❖ 초기 외부 형상 연구에서는 복잡한 Assembly 기능이 필수는 아님.
- ❖ Part::Feature, Compound, Boolean 연산으로 먼저 구성할 예정

■ 연구 진행 단계

진행 단계

❖ 4단계: 재현 baseline 실험

⇒ 공식 CRM을 재현하는 전체 코드를 LLM에 제공하여 파이프라인의 성능 상한을 측정.
이 결과는 일반화 성공이 아니라 “정답 정보를 충분히 주었을 때 실행 가능한가”를 확인하는 실험으로 방향성 설정

진행 단계

❖ 5단계: 단계적 정보 제거

❖ 다음 정보를 순차적으로 제거

전체 생성 코드

- 구성요소별 코드
- 제작 순서
- 세부 좌표
- 단면 배치 정보
- 공통 CAD 원리만 유지

✓ 각 단계에서 성능 저하와 실패 원인을 측정

■ 연구 진행 단계

진행 단계

❖ 6단계: 일반화 평가

- 치수와 배치가 다른 수송기 날개
- 보지 않은 항공기
- 전투기
- 무미익기
- 복엽기
- eVTOL

⇒ 형상 종류별 생성기를 추가하지 않고, 공통적인 단면·Loft·Sweep·변환·접합·B-rep 검증 지식으로 확장되는지 평가

진행 단계

❖ 7단계: 메싱 적합성 평가

⇒ Salome에 입력되는지 확인 필요

■ Baseline과 Rule-base의 경계 (연구 방향)

룰베이스에 빠지지 않는 baseline

❖ CRM 전체 코드 제공은 초기에는 강한 base-line으로 작용할 수 있다. 하지만 다음 조건을 지키면 최종 Rule-base로 고정되는 것을 피할 예정

- CRM이라는 단어로 CRM 코드를 자동 선택하지 않음
- 항공기 종류별 `if/else` 생성기를 만들지 않음
- 좌표 숫자만 바꿔 새로운 항공기라고 주장하지 않음
- 코드 제거 자체를 학습이나 발전이라고 주장하지 않음
- 제거 후 발견된 공통 실패를 범용 지식·관측·검증 개선으로 연결
- 보지 않은 항공기 계열에서 별도 평가

⇒ 완성 예제를 제거한 뒤에도 LLM이 새로운 구성과 코드를 만들고, holdout 항공기에서 성능이 확인돼야 baseline이 일반화 지식으로 전환됐다고 판단

Any Questions?

이름 최준석

소속 한양대학교 기계공학과

이메일 chw10109@hanyang.ac.kr



*Intelligent Design
for Engineering
Applications Lab*

